

SEMANA 3 ACADEMIA MONTERREY

- Threading, manejo de archivos y serialización

Los temas de threading, manejo de archivos y serialización en Java se tomaron en los días 1 y 2 de la semana 3 de la academia. Cada uno maneja diferentes conceptos:

- **Manejo de archivos:**

Se habló sobre los temas relacionados a Java I/O y NIO.2.

1. Java I/O es la API original de gestión de archivos que se introdujo en Java 1.0 y se basa principalmente en la clase `java.io.File`. Sus características relevantes incluyen el mencionar que es funcional para operaciones sencillas con archivos y directorios, pero presenta limitaciones en el manejo de errores, carece de características avanzadas como la gestión de enlaces simbólicos o el movimiento atómico de archivos y realiza operaciones de I/O bloqueantes.
2. Java NIO.2 se introdujo en Java 7 como una mejora respecto a la API original. Incorpora la interfaz `Path` junto con clases de utilidad como `Files`, `Paths` y `FileSystems`.
Admite operaciones no bloqueantes, sistemas de archivos personalizados, movimientos atómicos, control de enlaces simbólicos y un manejo avanzado de excepciones.

Las instancias de `File` se crean utilizando `new File(...)`, mientras que las instancias de `Path` se crean mediante métodos de fábrica (`Path.of(...)`, `Paths.get(...)`, `FileSystems.getDefault().getPath(...)`). A continuación podemos ver el ejemplo en código:

```
// Legacy File to Path
```

```
File legacyFile = new File("data/stripes.txt");  
Path pathFromFile = legacyFile.toPath();
```

```
// Path to Legacy File
```

```
Path modernPath = Path.of("data/stripes.txt");  
File fileFromPath = modernPath.toFile();
```

Estos dos códigos reflejan la conversión de la API original a la extensión Path, y viceversa.

También se habló sobre la manipulación de los paths utilizando las funciones `resolve()`, `relativize()`, y `normalize()`. A continuación se presentan códigos ilustrando estos conceptos:

```
Path p1 = Path.of("usr/home");
Path p2 = Path.of("docs/file.txt");
System.out.println(p1.resolve(p2)); // Output: usr/home/docs/file.txt
```

En este ejemplo, la función `resolve()` combina dos paths juntos.

```
Path p1 = Path.of("fish.txt");
Path p2 = Path.of("friendly/birds.txt");
System.out.println(p1.relativize(p2)); // Output: ../friendly/birds.txt
```

En este ejemplo, la función `relativize()` calcula el path relativo para viajar desde el path A al path B. Las rutas deben ser del mismo tipo, es decir, completamente relativas o absolutas.

```
Path p = Path.of("armadillo/../shells.txt");
System.out.println(p.normalize()); // Output: shells.txt
```

La función `normalize()` en este ejemplo remueve los elementos redundantes en los directorios hacia los archivos, como por ejemplo `.` (current directory) o `..` (parent directory).

La pregunta sobre cuándo usar I/O contra NIO.2 tiene que ver con que tan moderna es la base de códigos a trabajar. Para aplicaciones antes de Java 7, se debe trabajar con parámetros File de I/O, mientras que en el desarrollo de aplicaciones modernas es recomendable usar NIO.2 ya que provee mejor rendimiento y sintaxis de código más limpias, además de que se integra con Java Streams de manera fluida.

- **Threading**

En Java 21, el término "hilos" (threads) se refiere a rutas de ejecución concurrentes.

1. Hilos de plataforma (hilos tradicionales): Hilos pesados gestionados directamente por el sistema operativo subyacente. Cada hilo de plataforma reserva una cantidad significativa de memoria (típicamente entre 1 y 2 MB por stack).
Cuando un hilo de plataforma realiza una operación de I/O (por ejemplo, esperar el resultado de una consulta a la base de datos o una respuesta de red), se bloquea y desperdicia capacidad de hilos del sistema operativo.
2. Hilos virtuales: Hilos ligeros que operan en modo usuario y son gestionados directamente por la Máquina Virtual de Java (JVM), en lugar de por el sistema operativo. Requieren asignaciones de memoria mínimas (alrededor de 2 KB, frente a los megabytes de los otros). Un hilo virtual se "monta" dinámicamente sobre un hilo de

plataforma del sistema operativo (denominado hilo portador o carrier thread) para ejecutar tareas de CPU. Cuando se encuentra una llamada de I/O bloqueante, se "desmonta" automáticamente, liberando así el hilo portador para que pueda atender a otros hilos virtuales hasta que finalice la operación de I/O.

En resumen, los hilos de plataforma son manejados por el sistema operativo, mientras que los hilos virtuales se manejan mediante la JVM, además de que sus pesos de memoria son pesados y ligeros, respectivamente.

Los hilos virtuales destacan en cargas de trabajo concurrentes con gran intensidad de I/O, como por ejemplo:

1. Microservicios y API web: Procesamiento concurrente de miles de solicitudes HTTP entrantes sin agotar los grupos de hilos.
2. Bases de datos y servicios externos: Gestión de llamadas de red de larga duración, consultas a API externas u operaciones de lectura/escritura en bases de datos.
3. Computación en la nube y serverless: Optimización del uso de recursos en entornos contenedorizados.

- **Serialización:**

La serialización en Java es el proceso de convertir un objeto en memoria (una instancia de una clase) en un flujo de bytes para que pueda almacenarse o transmitirse.

1. Serialización (escritura del objeto): Toma un objeto y lo convierte (por ejemplo, en un archivo binario .dat o .txt).
2. Deserialización (lectura del objeto): Toma el flujo de bytes del almacenamiento y reconstruye el objeto original en la memoria.

En Java, una clase debe implementar la interfaz marcadora `java.io.Serializable` para indicar que sus objetos pueden ser serializados. La serialización se utiliza para la persistencia de datos, es decir, guardar estados de objetos sin tener que mapearlos manualmente a formatos estructurados como CSV o XML.

```
import java.io.Serializable;

public class Gorilla implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private int age;
    private Boolean friendly;
    private transient String favoriteFood;
}
```

En este ejemplo, la clase Gorilla implementa Serializable con varios atributos. El proceso de serialización consistiría en crear una lista de objetos Gorilla y utiliza envoltorios de I/O, FileOutputStream dentro de un BufferedOutputStream que a su vez está dentro de un ObjectOutputStream, para escribir los objetos en un archivo llamado gorillas.data. El proceso de deserialización utilizaría FileInputStream, BufferedInputStream y ObjectInputStream dentro de un bucle `while` infinito. Una vez que el lector llega al final del archivo binario, Java lanza una excepción EOFException (End Of File Exception), indicando que todos los objetos han sido leídos por completo.

Otro aspecto importante sobre la deserialización es que si no se especifica la versión del UID, el proceso solo puede cumplirse si el ID de versiones es el mismo entre los objetos. Por ejemplo, añadir otro atributo a la clase Gorilla puede ocasionar una excepción si se elimina la línea:

```
private static final long serialVersionUID = 1L;
```

Debido a que de esta forma, Java permite deserializar archivos antiguos incluso si la estructura de la clase ha evolucionado.